

DBMS Min Project

Github link : https://github.com/shreekar2005/DBMS_Min_Project

This is a minor project for a 3rd-year Computer Science Database Management Systems (DBMS) course. The goal is to implement and understand the key layers of a database storage engine.

The details of the project problem are in `Assignment.pdf`.

For new people or people who want to work on this project:

1. `toydb` is the name of our **Database**.
2. `pflayer` is the **Page File Layer**. This is the lowest level, responsible for reading/writing pages from/to disk and managing the buffer pool.
3. `splayer` is the **Slotted Page Layer**. This layer sits on `pflayer` and implements a slotted page structure to manage variable-sized records within a page.
4. `amlayer` is the **Access Method Layer**. This layer also sits on `pflayer` and implements a B+ Tree index for efficient data retrieval.

Project Structure

- `Makefile` : Top-level makefile that proxies to `toydb/Makefile`.
- `README.md` : This file.
- `Assignment.pdf` : The project assignment details.
- `data/` : Contains sample `.txt` data files used to populate the database.
- `Reports/` : Contains project reports :
 1. `About Slotted Paged file.pdf` : where we discussed about how we planned structure of slotted paged file.
 2. `Report_Objective1.pdf` : where we talk about LRU MRU in buffer (implementation and comparison)
 3. `Report_Objective2.pdf` : where we talked about how we implemented slotted page related functions
 4. `Report_Objective3.pdf` : where we talked about how we create index for file and create indices in bulk.
- `toydb/` : The complete ToyDB source code.

toydb Directory Structure

The `toydb/` directory contains the complete source code for the ToyDB storage engine. The structure is refactored into modular layers.

- `amlayer/` : Contains the Access Method (B+ Tree) Layer.
 - `include/` : Headers for the AM layer (e.g., `am.h`).
 - `src/` : Source files for the AM layer (e.g., `am.c`, `aminsert.c`) and its tests.
 - `Makefile` : Makefile to build the AM layer and its tests.
- `pflayer/` : Contains the Page File (Buffer Manager) Layer.
 - `include/` : Headers for the PF layer (e.g., `pf.h`, `pfbuf.h`).
 - `src/` : Source files for the PF layer (e.g., `pf.c`, `pfbuf.c`) and its tests.
 - `Makefile` : Makefile to build the PF layer and its tests.
- `splayer/` : Contains the Slotted Page Layer.
 - `include/` : Headers for the SP layer (e.g., `sp.h`, `spscan.h`).
 - `src/` : Source files for the SP layer (e.g., `sp.c`, `spscan.c`) and its tests.
 - `Makefile` : Makefile to build the SP layer and its tests.
- `main.c` : The main C entry point for the `toydb.out` executable (future query processor).
- `Makefile` : The main makefile for building the entire `toydb.out` executable by combining all layers.
- `am.pdf` : Documentation for the Access Method layer.
- `pf.pdf` : Documentation for the Page File layer.

Build and Run Commands for toydb

Dependency installation for Ubuntu

1. **Update Package Lists:**

```
sudo apt update
```

2. **Install Dependencies:** This command installs the build tools (`make`) and the C compiler (`cc / gcc`).

```
sudo apt install build-essential
```

Dependency Breakdown

- `build-essential` : Installs `make` and the `gcc` compiler, which are required to build the project from the `Makefile` s.
-

Test Commands

This is the primary way to test the functionality of each layer. These commands build and run specific test executables. *I also mentioned to which Assignment Objective a particular test command belongs.*

- `make testall`: Runs all tests from all layers (`am`, `pf`, and `sp`).
- **PF Layer Tests:**
 - **(Objective1)** TEST : compare LRU and MRU strategies for buffer

```
make pf_testbuf
```

- `make pf_testbuf_generategraph`: Runs the main `testbuf` from the `pf` layer and generate graph.
 - To generate graph you need these python packages :
- ```
pip install pandas matplotlib
```
- `make pf_testpf`: Runs the main `testpf` from the `pf` layer .
- `make pf_testhash`: Runs the `testhash` from the `pf` layer .

- **SP Layer Tests:**
  - **(Objective2)** TEST : convert .txt file to .tdb file

```
make sp_testconvert
```

- **(Objective2)** TEST : find matching records in .tdb file

```
make sp_testfind
```

- **(Objective2)** TEST : insert records in .tdb file

```
make sp_testinsert
```

- `make sp_testdelete`**(Objective2)** TEST : delete records from .tdb file

```
make sp_testdelete
```

- **(Objective2)** TEST : scan all records from .tdb file

```
make sp_testscan
```

- **(Objective2)** TEST : give analysis of .tdb file use

```
make sp_testanalyze
```

- `make sp_testsp`: Runs the main `testsp` from the `splayer` .

- **AM Layer Tests:**
  - **(Objective3 Task1)** TEST : create index for .tdb file in single function call

```
make am_testcreateindex
```

- **(Objective3 Task2)** TEST : insert entry in .tdb one by one and also insert index for that entry in index file.

```
make am_testcreateindexincremental
```

- **(Objective3 Task3)** TEST : insert indices in bulk for sorted keys

```
make am_testbulkload
```

- `make am_test1`: Runs `test1` from the `am` layer .
- `make am_test2`: Runs `test2` from the `am` layer .
- `make am_test3`: Runs `test3` from the `am` layer .

---

## The Layered Architecture Philosophy of ToyDB

The `toydb` project is split into three distinct layers to enforce a clean separation of concerns, mimicking a real-world database storage engine.

- **toydb/pf1ayer (Page File Layer)**: This is the **foundation** of the database. Its only job is to manage files on disk and provide a buffer pool (an in-memory

cache of disk pages, likely using an LRU and MRU policy). It provides an API to get a page (e.g., `PF_GetNextPage`), release a page (`PF_UnfixPage`), etc. It knows nothing about the *content* of the pages; to the `pflayer`, a page is just an opaque block of 4096 bytes.

- **toydb/splayer (Slotted Page Layer):** This is a **client** of the `pflayer`. It requests new pages from `pflayer` and then formats them with a "slotted page" structure. This structure allows it to store, delete, and manage variable-sized records efficiently within a single page. It provides an API to insert a record, delete a record, etc.
- **toydb/amlayer (Access Method Layer):** This is also a **client** of the `pflayer`. It requests pages from `pflayer` to build and maintain a B+ Tree index. The nodes of the B+ Tree are stored within these pages. This layer provides a high-level API for key-based operations: insert a key-value pair, delete by key, and scan for a range of keys.

This separation allows the `pflayer` (the most complex part) to be developed and tested independently. The `amlayer` and `splayer` can then be built on top, trusting that the `pflayer` will correctly handle all file I/O and buffering. The final `toydb.out` executable links all three compiled layer objects (`pflayer.o`, `splayer.o`, `amlayer.o`) together with `main.c` to create the full program.